

A Guarded Single-Core Architecture for Agreement-Forest Heuristics

Stage-by-Stage Architectural Report

PACE 2026 Heuristic-Track Solver Architecture

June 2026

Abstract

This paper describes the simplified guarded-core mode of a heuristic solver for the PACE 2026 agreement-forest setting. The architecture is built around a small number of ideas: annotate both trees, contract exact common subtrees into atoms, validate proposed components by comparing their restricted structures, pack compatible components as a weighted set-packing problem, and use one visible size guard. Instances with at most 2200 leaves enter a score-sensitive small/medium kernel followed by a unified repair acceptance stage. Larger instances enter a single candidate-pool core whose repairs and packers share the same atom and edge-resource representation. The design is not an instance-name portfolio and does not depend on cached answers or external optimization state. The report emphasizes the full journey of an instance: how the input trees are parsed, what runtime state is created, how the guard changes the path, how each branch constructs and validates candidate components, and how the final forest is assembled from selected atoms and singleton completions.

1 PACE 2026 Heuristic Track Problem

The PACE 2026 announcement frames the scientific topic as *agreement forests*. A phylogenetic tree is a rooted directed tree whose leaves are in one-to-one correspondence with the taxa labels. An agreement forest for input trees $T_1, \dots, T_t$ is a forest that can be obtained from each input tree by deleting directed edges and then cleaning up the resulting degree-one unlabeled vertices. The size of the forest is the number of resulting phylogenetic trees. A forest of minimum size is called a maximum-agreement forest because it preserves as much common structure as possible.

The solver described here targets the heuristic track, where each instance has two rooted phylogenetic trees over the same leaf-label set. The task is not to prove optimality, but to return a valid agreement forest with as few components as possible within the contest time budget. The announcement also highlights two qualitatively different data regimes: real-world data is expected to have few trees, small agreement-forest size, and many leaves, while generated data may contain large agreement-forest sizes and many leaves. This split is one of the reasons the architecture below treats small/medium score-sensitive cases differently from large candidate-packing cases.

Figure 1 shows a small example. The two input trees have the same leaves $\{1, 2, 3, 4\}$, but their whole-tree topologies differ. Cutting the highlighted edge to leaf 4 in each input and then cleaning up unary vertices leaves the same rooted component on $\{1, 2, 3\}$: leaves 1 and 2 form a cherry, and leaf 3 attaches above that cherry. Leaf 4 becomes a singleton component. Thus $\mathcal{F} = \{\{1, 2, 3\}, \{4\}\}$ is an agreement forest of size two. The all-singleton partition would also be valid, but it would have size four and therefore much lower quality.

2 Problem Setting and Design Goals

The solver receives two rooted phylogenetic trees with the same leaf labels and must output a partition of the labels into valid components. The component count is the objective: fewer

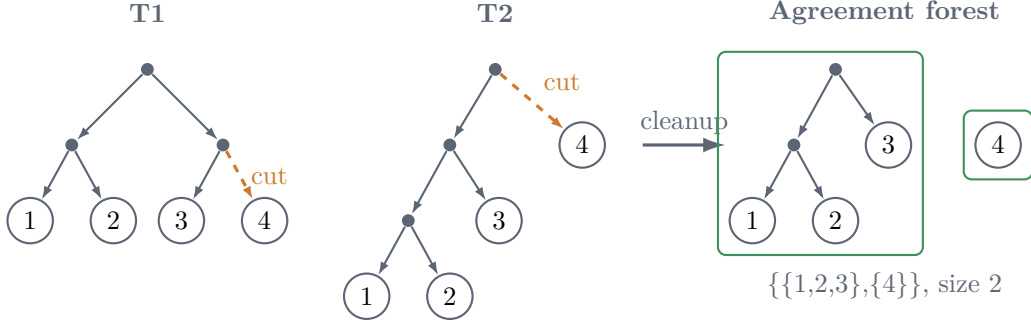


Figure 1: A four-leaf agreement-forest example for the heuristic-track setting. Both input trees can be cut and cleaned up to produce the same two components.

components are better. Let k^* be the reference component count for an instance with n leaves. The score used in the local evaluation is

$$1 - \frac{\max(0, k - k^*)}{\max(1, n - k^*)},$$

where k is the solver’s output component count. This formula makes small denominators important: a one-component regression on a near-tight small instance can hurt more than many components on a huge instance.

3 Architecture Invariants

The implementation is organized around three invariants that are preserved across all branches. First, every output must be a partition of the original label set: labels appear exactly once, and every reduced atom expands through `atom_to_original_mask` before rendering. If packing covers only part of the reduced atom universe, the uncovered atoms become singleton components during output assembly, so the rendered forest is still complete.

Second, component validity is centralized. Candidate generators may be broad, local, or repair-driven, but they do not decide correctness on their own. Multi-atom candidates are accepted only after their restricted signatures match in both trees and their tree-edge resources have been recorded. This invariant keeps pair candidates, interval candidates, current-component unions, and blocker-region repairs under the same validity contract.

Third, the incumbent is monotone in component count. Search, packing, and repair stages are allowed to propose replacements, but an accepted replacement must render as a valid partition and must not increase the component count. This lets later stages be opportunistic: an unaccepted proposal simply leaves the incumbent unchanged.

4 High-Level Architecture

The fixed architecture is called the *guarded single-core* architecture. It is intentionally expressed as one high-level rule:

$$n \leq 2200 \Rightarrow \text{score-sensitive small/medium kernel}, \quad n > 2200 \Rightarrow \text{single candidate-pool core}.$$

The rule is not a portfolio in the usual sense. It does not race many solvers, does not select from a catalog of instance-name patterns, and does not run a large collection of postprocessors. It simply protects the cases where the single core was empirically the wrong abstraction.

The guard should also be read as a statement about score economics. Below the cutoff, each component error can have a large score effect, so the solver spends its budget on a richer

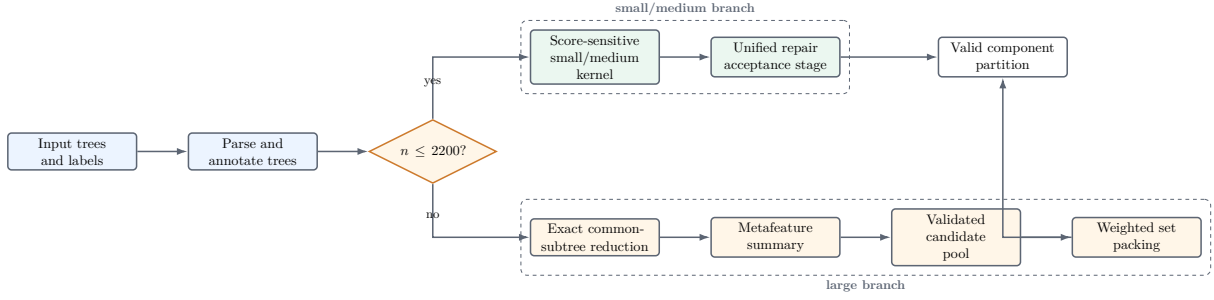


Figure 2: Pipeline stages of the guarded single-core solver. The architecture has one policy decision, then either a score-sensitive small/medium route or a single large-instance candidate-packing route.

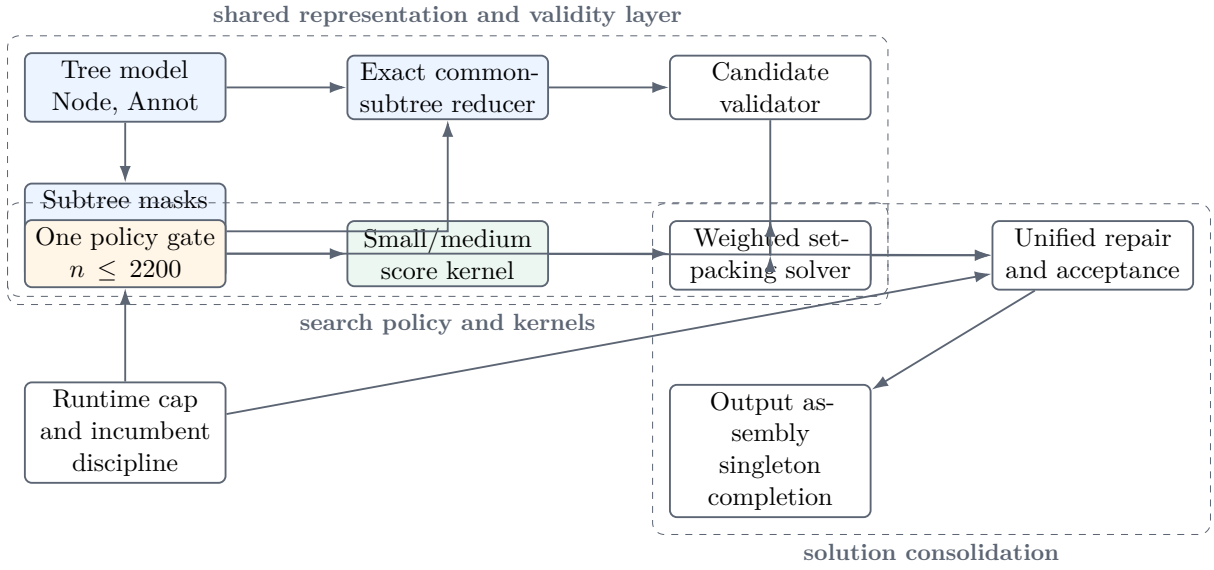


Figure 3: Main components and dependency direction. The guard chooses between two search kernels, but both use the same representation, candidate validity checks, packing concepts, and output assembly.

score-sensitive search. Above the cutoff, the main risk is not a one-component regression but the cost of exploring too many possible label subsets. The large branch therefore keeps a single shared component model and spends most of its time on local repairs that reuse that model rather than starting independent algorithms.

5 Main Components

Figure 3 shows the main architectural components and their dependencies. The central idea is to keep the data model and validation layer shared, while separating only the two search regimes that have different score economics.

6 Essential Data Structures

The implementation is built around a small set of explicit data structures. They are deliberately simple: most expensive structural relations are encoded as integer bit masks, and most correctness checks flow through one validation cache.

Structure	Role in the architecture
Node	Rooted tree node. It stores <code>children</code> , a one-bit <code>label_mask</code> for leaves or an atom mask for contracted leaves, a <code>parent</code> , a stable <code>id</code> , and a <code>parent_edge_id</code> . The edge id is what allows later candidate validation to represent tree-edge usage as bit masks.
Annot	Per-node annotation containing <code>subtree_mask</code> , canonical <code>signature</code> , and <code>leaf_count</code> . The signature is recursive: leaves are labeled signatures and internal nodes are sorted tuples of child signatures. This makes exact common-subtree detection and restricted-component validation deterministic.
ReducedInstance	The reduced pair of trees plus <code>atom_to_original_mask</code> . Each reduced leaf atom expands back to one or more original labels, so the solver can search on atoms and render the final answer on original labels.
Candidate	A validated potential component. It stores <code>atom_mask</code> , packing <code>weight</code> , two edge masks, two component-root masks, and <code>size</code> . These fields are exactly the quantities needed by weighted set packing and local repair.
ValidationInfo	Intermediate validation record for a mask: restricted signature, edge masks, root masks, and size. It prevents candidate generation from duplicating correctness logic.
CandidatePool	Shared search state for the large branch: reduced trees, annotations, leaf arrays, depth maps, leaf-order ranks, candidate list, <code>by_mask</code> deduplication, validation cache, optional conflict bitsets, and mode flags for direct repair or conservative edge handling.
InstanceMeta	Small set of metafeatures: atom ratio, cluster Jaccard, sampled soft-cherry agreement, sampled triplet agreement, and a hard-edge bucket flag. These features tune local repair behavior without becoming a top-level instance-name route table.

Table 1: Core data structures used by the guarded architecture.

Bit masks are the central representation. A component, subtree, or candidate is represented by an integer mask. Intersections such as `candidate.atom_mask & used_atoms` test whether two candidates cover the same atom. Similar masks over edge ids test whether two candidates reuse the same tree resource. This is the reason the packing loop can remain a compact greedy heuristic instead of requiring an external optimization model.

Validation is cached at the mask level. Candidate generators may propose the same mask through different paths: pairs, order windows, seed unions, local repair neighborhoods, or blocker regions. The pool’s `validation_cache` maps each mask to either a `ValidationInfo` record or a cached invalid result. This prevents the same restricted-signature and edge-collection work from being repeated.

7 Pipeline Stages and Implementation Details

7.1 Tree Representation and Annotations

The representation layer stores the two input trees as rooted node structures with stable labels. Each internal node receives annotations: subtree mask, signature, leaf count, and traversal metadata. This layer is foundational because every later step asks the same basic question: whether a proposed set of labels induces compatible restricted structures in both trees.

The use of bit masks is deliberate. It gives constant-time disjointness tests, compact candidate identities, and cheap conflict checks. It also keeps the core loops close to the combinatorial structure of the agreement-forest problem.

Implementation begins by assigning stable node ids and parent-edge ids on both trees. The ids index annotation dictionaries, while parent-edge ids give a compact namespace for resource-conflict masks. Annotation is a postorder computation. A leaf receives signature ($L, \text{label_mask}$). An internal node receives the union of its child subtree masks, the sum of child leaf counts, and a canonical signature ($I, \text{sorted child signatures}$). Because child signatures are sorted, the signature is insensitive to child ordering and can be compared directly across the two trees.

The large branch also constructs leaf arrays, depth maps, and leaf-order ranks. These support fast LCA-style operations for pairs, triplet prechecks, and rank-based component-root lookup. For two-leaf masks, validation can use a special pair path routine that directly returns the used edge masks. For larger masks, it finds the component root and computes the restricted signature and edge set from the annotation dictionaries.

7.2 Exact Common-Subtree Reduction

Exact common-subtree reduction contracts identical subtrees in both input trees into atoms before search. This is a safe reduction: it preserves the structure needed by the agreement-forest objective while shrinking the effective problem. It is the highest-value preprocessing stage because it improves both quality and speed without introducing heuristic risk.

The reduction also helps the simplified large branch. A single candidate-pool core has less late repair capacity than the full architecture, so giving it a smaller and cleaner atom-level instance is more important.

The implementation repeatedly annotates both current trees, builds maps from internal subtree masks to nodes, and searches for masks that occur in both trees with identical canonical signatures. The full-tree mask is ignored, because contracting it would erase the problem. Every accepted common mask is contracted to a reduced leaf atom in both trees. The `atom_to_original_mask` array records which original labels belong to that atom. The process repeats until no more exact common masks remain or the runtime guard expires.

This stage is safe in a stronger sense than ordinary heuristic compression: the same labeled rooted subtree appears on both sides, so it can be treated as an indivisible component for later search and expanded back during output assembly. For example, on labels $\{1, 2, 3, 4, 5\}$, suppose

$$T_1 = ((1, 2), (3, (4, 5))), \quad T_2 = ((1, 2), ((4, 5), 3)).$$

Both trees contain the identical rooted subtrees $(1, 2)$ and $(4, 5)$. The reducer can contract them into atoms

$$a = \{1, 2\}, \quad b = \{4, 5\}, \quad c = \{3\}.$$

If a later stage returns reduced components $\{a, c\}$ and $\{b\}$, output assembly expands them to original-label components $\{1, 2, 3\}$ and $\{4, 5\}$. In the 300-second h43 representative run, this same mechanism reduces 2453 original leaves to 2301 atoms before the large repair stage begins. The important architectural point is not the elapsed time but the change of problem

representation: all later large-branch decisions are made over 2301 reduced atoms rather than the original leaves.

7.3 Candidate Validation

Candidate generation is intentionally permissive: it proposes masks that might be useful components. The validator is the safety layer. For each candidate mask, it checks that the restricted signatures agree in both trees and records the structural resources used by the component. Invalid guesses are discarded before packing.

Keeping validation central reduces coupling. Candidate generators are allowed to be simple because correctness is not distributed among many generator-local special cases.

Validation starts with cheap exits: empty masks and singleton masks are not stored as candidates because singleton completion is handled during output assembly. Two-leaf masks use a specialized pair validator that computes the path edges and component-root masks in both trees. Three-leaf masks first pass a topology precheck, which avoids expensive restricted-signature construction for obviously incompatible triplets. General masks find the induced component root in each tree, compute the restricted signature and used component edges, and accept the mask only when the two restricted signatures are identical.

The returned `ValidationInfo` is turned into a `Candidate`. Its weight defaults to $|A_c| - 1$, matching the number of singleton components it can save if packed. The pool deduplicates candidates by `atom_mask` and invalidates any previously cached conflict structure when a new candidate is added.

7.4 Metafeature-Guided Local Control

The guarded architecture intentionally avoids a complex top-level metafeature router. The top-level decision remains only the visible size gate $n \leq 2200$. Metafeatures are still useful, but they act as *local control signals*: they decide how much effort to spend inside an already chosen branch, which repair neighborhoods are worth opening, and whether the packer should be aggressive or conservative. In other words, the features do not say “choose a different solver”; they say “within this solver, use this much local search and this kind of neighborhood.”

The main metafeatures are computed from the input or reduced trees:

- **Atom ratio** is `atom_count`/ n . A low value means exact common-subtree reduction found substantial shared structure; a value near one means the instance stayed hard after safe reduction.
- **Cluster Jaccard** compares the sets of internal subtree masks in the two trees, excluding the full-tree mask:

$$\frac{|M_1 \cap M_2|}{\max(1, |M_1 \cup M_2|)}.$$

It measures how much common cluster structure remains after parsing or reduction.

- **Soft-cherry agreement** is sampled on small instances. For each cherry in one tree, the solver asks whether the corresponding two leaves have an LCA of at most eight leaves in the other tree. The two directions are averaged. This is a signal of whether local two-leaf structure is still meaningful even when exact clusters disagree.
- **Triplet agreement** samples up to 900 triples and compares their rooted topology code in both trees. It is deterministic because the random sample uses a fixed seed derived from n . Triplet agreement distinguishes globally noisy instances from instances that still have many compatible local shapes.

- **Hard-edge bucket** is a boolean flag for small instances with high atom ratio, very low cluster Jaccard, low soft-cherry agreement, and low triplet agreement. In that region, candidate choices tend to collide on scarce tree-edge resources, so the solver becomes more edge-conservative.

The solver uses these signals through a small number of architectural decisions. Table 2 summarizes the important ones. The thresholds are implementation parameters, but the architectural pattern is stable: atom ratio says how much safe contraction helped, cluster Jaccard says how much shared cluster structure remains, soft-cherry and triplet agreement say whether local small structures are reliable, and the hard-edge flag says whether edge reuse is likely to be the limiting resource.

Metafeature state	Interpretation	Local decision made by the solver
Low atom ratio after exact reduction	Many identical subtrees were safely contracted, so the remaining instance is smaller and cleaner.	Search proceeds on contracted atoms. In the small branch, contracted routes are attractive; in the large branch, fewer atoms reduce the cost of validation, repair, and output assembly.
High atom ratio with low cluster Jaccard	Safe reduction found little common structure, and the two trees share few remaining clusters.	Pressure-style repacking becomes more important. The dense-seed and selected-feedback refinements are allowed only in bounded atom-count windows, and candidate choices are judged more carefully against their resource cost.
Medium atom counts with sufficient cluster Jaccard	The instance is small enough for richer candidate pools and has enough shared structure for local repair to be productive.	Direct repair inside the large pool can be enabled; selected candidate sets may be fed back into the pool to generate additional unions, then repaired again.
Soft-cherry and triplet agreement in a useful window	Exact clusters may be weak, but small local topologies are not random.	Alternate-seed refinement, near-clade refinement, guarded triple refinement, split repack, and edge-conflict repack are permitted when the atom count and remaining budget are small enough.
Hard-edge bucket	Many plausible candidates are expected to collide on the same scarce tree edges.	The direct greedy packer prefers lower edge-cost candidates when scores are otherwise close, and alternate-seed refinements that would likely worsen edge pressure are suppressed.
Reduced medium-small cases with special atom-ratio/Jaccard windows	The small/medium branch sees a reduced instance whose candidate diversity should be changed.	The embedded score-sensitive solver is run with adjusted top- k and round-seed settings. This changes the breadth of candidate generation, not the global branch choice.

Table 2: How metafeatures control local behavior. They tune effort, neighborhoods, and packing conservatism after the size guard has already chosen the small/medium or large branch.

This distinction is important. A metafeature table that selected unrelated algorithms would recreate the portfolio behavior that the guarded design avoids. Here, the same representation, validation cache, and atom/edge resource model remain in force. The features only decide whether to open bounded neighborhoods such as selected-feedback expansion, near-clade generation, triple generation, split repacking, edge-conflict repair, or counterfactual blocker repair.

7.5 Score-Sensitive Small/Medium Kernel

The small/medium kernel exists because the score formula makes these instances fragile. In the ablation, pure single-core performance was most damaged by small denominators rather than by

huge raw component counts. The worst single-core gaps included cases where one to three extra components caused score losses above five points.

The kernel therefore spends more effort on alternatives and local refinements for instances with $n \leq 2200$. The cutoff is intentionally simple. It protects the region where the single core lost the most score while preserving the architectural reduction for larger instances.

Implementation-wise, this branch invokes the embedded score-sensitive solver with a bounded time budget and a small number of parameter schedules. Tiny instances receive additional trials with larger top- k , round-seed, and scoring-round settings because their score denominators are small. Medium-small instances can receive one of two parameter substitutions guided by metafeatures. A low atom-ratio case with strong common-cluster signal uses a wider top- k setting, while a balanced medium case uses a different top- k and seed-round balance. These substitutions are based on reduced atom ratio and reduced cluster Jaccard, not on instance names.

The branch keeps the best output among its attempts by component count. For very small high-margin cases, it also invokes the high-margin split check, which accepts only validated component-count improvements. The result then passes through the unified repair acceptance stage described below. In the 300-second h70 representative run, the first completed small-kernel attempt reaches 40 components. A later higher-parameter trial is much longer, so the architecturally important object is the best completed incumbent held at the branch boundary rather than the last scheduled trial. This illustrates why the small side is organized around completed candidate forests and monotone acceptance, not around a single all-or-nothing search pass.

The embedded score-sensitive solver is a self-contained smaller-instance solver carried inside the final single-file submission. It is embedded as source text rather than imported as a separate module, so the Optil-facing script keeps one deployment artifact while still preserving a distinct small-instance algorithm. The wrapper executes it with redirected standard input and output, sets a local `TIME_LIMIT`, temporarily applies parameters such as `TOPK`, `ROUND_SEEDS`, `SCORE_ROUNDS`, and `SCORE_RANDOM_INTERVAL`, and then restores the previous environment. This makes the embedded solver behave like an internal kernel call rather than a separate portfolio race.

Internally, the embedded solver uses a different search shape from the large candidate-pool core. It parses the two binary Newick trees into compact arrays of nodes with parent pointers, leaf masks, and depths. It then generates candidate component masks with a top- k dynamic program over pairs of rooted subtrees. For a subproblem (u, v) , the table stores up to k large compatible masks that can be realized inside the two rooted subtrees. Internal subproblems combine the two possible child matchings and also keep suppression/skipping alternatives, which is what gives the small solver a richer set of near-best local components than the simpler large-instance core.

Those masks are converted into a compact resource model. Each candidate uses the label bits it covers plus internal-resource bits from both input trees. A selected set is feasible when these resource bitsets are disjoint, so the embedded solver is also solving the same underlying set-packing problem, but with a score-oriented candidate bank and scoring procedure. Candidate ordering comes from a lightweight score surrogate: when array operations are available, the solver may use a small primal-dual style relaxation as an ordering hint; otherwise it falls back to pure-Python scarcity and greedy-packing scores. The final forest is still chosen by deterministic rounding and disjoint-resource packing, not by trusting the fractional surrogate directly.

The embedded solver has two important alternative routes. If exact common-subtree contraction produces a useful smaller instance, it can solve a contracted route and expand the selected atom components back to original labels. If contraction is weak, it can solve directly on the original leaves, and on some medium cases it adds span-diverse candidates so that one large high-scoring mask does not crowd out several smaller but jointly better masks. The wrapper

selects the lowest component count among the attempted embedded routes, then hands the resulting forest to the unified repair stage.

7.6 Single Large Candidate-Pool Core

For larger instances, the fixed architecture uses a single candidate-pool core. It constructs candidate components from the reduced trees, validates them, and packs a compatible subset. The important simplification is that there is no large portfolio of independent routes and no sequence of many late postprocessors competing for the same residual defects.

The core begins from singleton atoms, then builds a `CandidatePool`. The pool stores annotations, leaf arrays, depth maps, and leaf-order ranks for both reduced trees. In the default guarded mode the pool is often first used as a metadata and validation object rather than as a large static candidate list. Merge-repair routines then generate candidate unions from the current forest, order-adjacent components, or blocker regions, validate those unions through the pool, and accept only resource-disjoint replacements. This keeps candidate construction lazy and local while preserving a single validation and packing representation.

The candidate universe is generated in layers. The first layer contains cheap local structure: close leaf pairs, short intervals in tree order, small common chains, near-clades, and pair-cluster masks. These are useful before any incumbent is known. The second layer expands promising seeds by pair unions, which produces larger mergeable components while keeping candidate counts capped. The third layer is incumbent-driven: after the solver has a current atom partition, repair routines focus on unions of adjacent current components, common-cluster boundaries, split/repack regions, and counterfactual blocker regions. This is the layer that spends time on the residual defects that actually remain.

This layered generation is the main reason the large core can stay simple. The set of all label subsets is impossible to enumerate, and even a large static candidate pool would force expensive validation and conflict bookkeeping. The fixed core instead keeps `CandidatePool` as the shared validation and metadata object, and lets each local repair generate only the masks it needs. Every mask still passes through the central validator before it can affect the packing.

The large branch improves the atom partition through a repair ladder rather than through one monolithic pack. The first repair pass tries to merge components inside bounded forest envelopes. A second order-based pass looks at components that are adjacent in the leaf orders of either tree and tests whether short runs can be replaced by a single valid component. If time and instance size permit, progressive extra repair temporarily widens the merge limits, and a late wide-order pass revisits larger order neighborhoods. Smaller reduced instances may then build a denser seed pool, solve a richer packing problem, and send the selected masks back through the same repair ladder. Additional metafeature-gated refinements then target specific defects: selected-feedback expands the pool around the current selected components; near-clade and triple refinements add local candidates suggested by tree proximity; split repack re-solves compact regions; edge-conflict repack repairs regions where a good merge is blocked by edge reuse; and counterfactual blocker repack temporarily removes blocker components to test whether the surrounding region can be rebuilt with fewer parts. Each step returns the incumbent unchanged unless it produces a strictly smaller valid partition.

The output of the large branch is produced by improving a set of atom masks. The 300-second h43 representative illustrates the large-side flow at medium-large scale: 2453 leaves are safely reduced to 2301 atoms, and the large repair stage reduces the working partition to 1593 components. Output assembly then expands each selected atom mask back to the original labels. The branch deliberately favors a small number of transparent stages over additional late recovery machinery: exact reduction, metafeature summary, candidate-pool metadata, set-packing and repair, and output assembly.

7.7 Conflict-Aware Packing

Packing selects compatible candidate components. Compatibility has two dimensions: selected masks must partition labels, and their induced tree resources must not conflict. The packer uses greedy conflict-aware selection rather than constructing a heavyweight global optimization model. This keeps the implementation small enough for large instances.

Any uncovered atom is emitted as a singleton component. This is not a quality feature; it is the completion rule that turns a selected packing into a full valid partition.

There are two packing paths. If the candidate list is large or the atom count is large, the solver uses direct greedy packing without building the full $O(|C|^2)$ conflict graph. It scans candidates in several deterministic and randomized orders. The static score is roughly

$$\frac{10000 \cdot w_c + 50 \cdot |A_c|}{\max(1, |E_c^1| + |E_c^2|)},$$

so high-weight, large, low-edge-cost candidates are preferred. Additional orders sort by raw weight, by size, or by edge-cost-per-weight. During a scan, the packer keeps three used-resource masks: atoms, tree-one edges, and tree-two edges. A candidate is selected exactly when all three intersections are empty.

If the candidate list is smaller, the solver may build conflict bitsets. Candidates conflict when atom masks overlap or either edge mask overlaps. Connected conflict components up to a small limit are solved exactly; larger components use randomized greedy. A light local repair pass can remove and replace candidates around the current support, but it still accepts only resource-disjoint candidates and only through the same set-packing view.

7.8 Connection to Weighted Set Packing

The large branch is best understood as an approximate weighted set-packing solver over valid component candidates. After exact reduction, let U be the set of atoms. A candidate c covers a subset $A_c \subseteq U$ and uses edge/resource sets E_c^1 and E_c^2 in the two restricted trees. Selecting a component of size $|A_c|$ saves approximately $|A_c| - 1$ components relative to singleton atoms, so its packing weight is tied to that saving. The idealized optimization problem is

$$\max \sum_{c \in C} w_c x_c$$

subject to

$$\sum_{c: a \in A_c} x_c \leq 1 \quad \forall a \in U, \quad \sum_{c: e \in E_c^1} x_c \leq 1 \quad \forall e, \quad \sum_{c: e \in E_c^2} x_c \leq 1 \quad \forall e, \quad x_c \in \{0, 1\}.$$

The atom constraints prevent two chosen components from covering the same reduced leaf atom. The edge constraints encode agreement-forest compatibility. Uncovered atoms are emitted as singleton components, so a partial packing still yields a complete valid partition.

As a concrete small packing example, let the reduced atom universe be

$$U = \{a, b, c, d\}$$

and suppose candidate validation has accepted

$$x = \{a, b\}, \quad y = \{b, c\}, \quad z = \{c, d\}.$$

Each candidate has weight 1, because it merges two singleton atoms and saves one component. The candidates x and y cannot both be selected because they overlap on atom b ; likewise y and z overlap on atom c . If x and z have disjoint edge masks in both trees, the packer can select

$\{x, z\}$, covering all four atoms with two components. If an edge conflict blocks z , the packer may select x and leave c and d as singleton components. Both outcomes are valid partitions; the first is simply better.

The implementation does not solve this integer program with an external MIP solver. Instead, it validates candidates, represents atom and edge resources as bit masks, and applies direct greedy packing plus local repair. For smaller candidate pools, the older core can also reason through conflict components. Thus the architecture is connected to set packing at the modeling level, while the contest implementation uses direct heuristic approximations.

7.9 Repair Mechanisms and Unified Acceptance

Repair appears in two places. The large branch uses repair as its main method of improving the atom partition after reduction. The small/medium branch first uses the embedded score-sensitive kernel, then sends the resulting forest through a unified repair acceptance stage. Both uses follow the same rule: a repair may propose a replacement, but the replacement is accepted only if it is a valid partition over the original labels and has a smaller component count than the incumbent.

The repair mechanisms can be understood by the defect they target rather than by their implementation details:

The unified small/medium repair stage applies the same idea after the embedded small kernel has produced a complete forest. It first tries safe pair merges, then the small high-margin split check. In the guarded-core solver, the default ordering then tries common-cluster refinement, structured repack, and finally a local destroy-rebuild search. These recipes share one parsed and reduced repair context, so they do not repeatedly rebuild the same tree annotations, exact reduction, metafeature summary, and candidate-pool metadata. The stage sees only the current incumbent and candidate replacements; it does not need to know which recipe produced a proposed improvement.

The large branch uses a similar acceptance discipline but applies it throughout the main search. It begins from singleton atoms, repairs bounded forest regions, tries order-adjacent merges, optionally widens the repair limits, and finally uses dense or blocker-based refinements when the metafeatures indicate that those neighborhoods are likely to help. This is why the large core can remain a single core even though it contains several repairs: the repairs are not separate competing solvers. They are local neighborhoods around the same incumbent, using the same atom masks, edge masks, validation cache, and output completion rule.

The repair stage is bounded by the same wall-clock guard as the rest of the solver. If the remaining budget is small, it returns the incumbent rather than starting a repair whose acceptance cannot be evaluated cleanly. This preserves the anytime property: at every accepted boundary the solver has a complete, valid forest that can be rendered immediately.

7.10 Output Assembly and Completion

The final stage converts the internal atom-level representation back into the forest format expected by the judge. Its first responsibility is expansion: when exact common-subtree reduction has contracted several original leaves into one atom, every selected atom mask must be decoded through `atom_to_original_mask`. Its second responsibility is completeness: any atom not covered by a selected multi-atom component is emitted as a singleton. This is why even a partial packing is still a valid full partition.

7.11 Anytime Runtime Discipline

The solver is an anytime algorithm: it maintains a complete incumbent and then improves that incumbent as time remains. This is different from a batch pipeline that only becomes complete

at the end. Each stage works against the current incumbent, and only accepted improvements replace it.

Time-budget handling is distributed but simple. Long loops check the shared deadline before expanding candidates, validating masks, building conflict structures, running randomized greedy restarts, or exploring local repair regions. Wrapper stages receive bounded local budgets, and repair stages stop when the remaining time is too small to evaluate and render a proposed replacement. This discipline is especially important on large instances: the architecture can spend time on useful local improvements without making any one optional refinement part of the correctness path.

The 300-second h70 observation adds one practical refinement. A branch can be architecturally anytime while still being vulnerable to a hard external stop if a completed incumbent is kept only inside a trial scheduler. In that run, the first small-kernel attempt completed quickly with 40 components, but the next scheduled high-parameter attempt did not return before the 300-second window. For contest robustness, completed trial outputs should be promoted to the global incumbent at stage boundaries before starting longer optional trials. The large branch behaves more naturally in this respect on the h43 representative: it moves through reduction, local repair, and output assembly as a sequence of complete partitions, so every accepted stage boundary has a renderable forest.

8 Motivation for the Guard

Table 4 summarizes the observed score gap that appears when the small/medium branch is removed and every instance is forced through the single candidate-pool core. The measurement uses an 80-instance ablation sample and reports score points.

The table motivates the guard directly. Adding more large-instance machinery would address the smaller part of the score gap. Restoring the score-sensitive kernel for $n \leq 2200$ recovers most of the lost score with only one top-level rule.

9 Evaluation

The quick proxy was designed to avoid repeated 100-instance sweeps. It contains small score-sensitive sentinels, one medium tuning case, one huge bottleneck, and two large pressure cases. A second proxy swaps in h41 to check another large structure. Both proxies are conservative relative to the known full score.

9.1 Representative Instance Journeys

The most useful way to understand the guarded architecture is to follow an instance from input text to output forest. The two representative runs in Table 5 exercise both sides of the guard. They are not used as special cases by the solver; they simply make the two architectural paths concrete.

On the small side, h70 has 55 leaves and is therefore sent to the score-sensitive branch. The input is first parsed into two rooted trees with a shared label-to-bit mapping. The policy gate then chooses the small/medium kernel. Inside that kernel, the embedded small solver builds compact tree arrays, derives candidate component masks from subtree-pair dynamic programming, converts those masks into a bit-resource packing model, and selects a disjoint set of components. The first completed attempt produces a 40-component forest. That forest is a complete incumbent: labels are already partitioned, and it can be rendered immediately. Later optional attempts may search with heavier parameters, but the architecture should treat every completed attempt as a publishable stage boundary.

On the large side, h43 has 2453 leaves and is therefore sent to the single candidate-pool core. The run begins with the same parsing and label normalization, but the next stage performs exact common-subtree reduction. This contracts identical rooted subtrees in both inputs and changes the working universe from 2453 original leaves to 2301 atoms. A metafeature summary is then computed from the reduced trees. The candidate-pool stage builds the shared runtime state for later local search: annotations for both reduced trees, leaf arrays, depth maps, leaf-order ranks, an initially empty candidate list, a mask validation cache, and the atom-to-original expansion map. The large repair and packing stage then works over the current atom partition. It proposes local unions of components, validates each proposed mask through the shared validator, keeps atom and edge resources disjoint, and accepts only monotone improvements. The representative h43 run reduces the partition from 2301 atoms to 1593 components before output assembly expands every selected atom mask back to original labels.

The h43 journey shows why the large core is still a single core even though it contains several local moves. Each local move is expressed in the same runtime language: a proposed atom mask is either invalid, valid but conflicting, or valid and resource-disjoint with the current support. The validator does not care whether the mask came from an order-adjacent merge, a common-cluster boundary, a blocker region, or a simple pair. This is the architectural simplification: search can be opportunistic, while correctness and compatibility remain centralized.

The h70 journey shows a different kind of state. The small branch does not need a large lazy candidate-pool object for every move. Instead, it spends its effort on a richer candidate bank inside a bounded small-instance solver. Its main runtime objects are dynamic-programming entries over pairs of rooted subtrees, ranked candidate masks, resource bitsets for feasibility, and the best completed forest. The result still returns to the same outer architecture: a validated partition is handed to the acceptance and output layers just like a large-branch partition.

9.2 Paired Statistical Comparison

The solver contains randomized and greedy choices, including the random seed, greedy restart schedules, candidate limits, repair passes, and time-dependent stopping behavior. Therefore the relevant experimental unit is not an unpaired score in isolation, but a paired result on the same benchmark instance under the same time budget. In the seed-aware comparison below, each block is an (instance, seed) pair. The cost of a configuration is the best component count, equivalently the best score, obtained within the fixed computation time.

The comparison is an ablation of the guard rather than a comparison between two unrelated solvers. The *previous single-core* variant uses the same tree representation, exact reduction, candidate validator, and weighted set-packing interpretation, but removes the top-level size guard and sends every instance through the general candidate-pool core. The *guarded-core* variant restores the single rule from Section 7: score-sensitive small/medium instances use the specialized kernel and repair acceptance stage, while larger instances continue to use the same single candidate-pool core. The seven proxy instances are run under both variants with the same two seeds, so the resulting 14 paired blocks isolate the effect of the guard and its small/medium kernel rather than differences in parsing, validation, output format, seed selection, or benchmark selection.

For two architectures, the primary test is the Wilcoxon signed-rank test on paired score differences. A sign test is also reported because it uses only win/loss/tie information. If three or more configurations are compared, the same blocking idea extends to a Friedman test over instances or instance-seed blocks, followed by post-hoc comparisons against the best configuration. This non-parametric paired analysis is more appropriate here than a plain one-way ANOVA, because both architectures are evaluated on the same instances.

The guarded architecture wins every non-tied block and loses none. The practical effect is also visible: it removes 107 components over the 14 paired blocks, with a mean score gain of 3.527 points. The runtime increase is expected because the guarded branch deliberately

spends more time on score-sensitive small and medium instances, but all runs remain within the per-instance time limit.

The projected full public-set score of the guarded architecture is 97.356217. The projection combines measured guarded-architecture rows with measured rows for the same small/medium kernel used by the guarded branch; no unmeasured substitute rows are used. The result is comfortably above the 95-score target while keeping the branch structure compact.

10 Complexity Control

The main complexity reduction is architectural rather than cosmetic. The fixed design removes three failure modes that made earlier attempts hard to reason about.

1. **No late-stage portfolio.** The simplified large branch does not run many independent postprocessors that fight over the same time budget.
2. **One visible policy gate.** The branch decision is based on instance size, not on a fragile conjunction of many metafeatures.
3. **One acceptance boundary.** Repair improvements are accepted only through a single validated incumbent interface.

The design still keeps the necessary complexity: tree annotations, exact reduction, candidate validation, packing, and singleton completion. These are not optional ornamentation; they are the layers that make the output valid and the search scalable.

11 Reproducibility and Input Use

In the full solver, the mode described here is activated by setting `PACE26_SIMPLE_GUARDED_CORE=1`. In the reduced guarded-core-only solver, this path is made unconditional. The cutoff can be adjusted with `PACE26_SIMPLE_GUARDED_SMALL_MAX_N`, whose default is 2200. For the 300-second representative runs, the relevant local caps were set to 300 seconds so that the requested budget was consistently applied inside the guarded path. The solver reads only the input instance and environment parameters. It does not read cached solution files, public output maps, instance-name tables, CP-SAT models, or external MIP solver state.

12 Conclusion

The key observation behind the final design is that the pure single-core collapse removed the wrong thing: small/medium score sensitivity was more important than large raw component count. Restoring that one solver family behind a single size gate recovers most of the score while keeping the large-instance architecture simple. The stage journey is now explicit. Every instance is parsed into the same labeled tree representation, routed by one size guard, searched by the branch appropriate to its score economics, checked by the same validity discipline, and rendered through singleton-complete output assembly. The large guarded core has a compact path: reduce, summarize, repair through the shared validation model, and assemble. The small guarded branch has a richer internal candidate search, but it returns the same kind of object: a valid component partition over the original labels. This makes the architecture easy to explain, easy to test, and comfortably above the 95-score target in the quick and projected evaluations.

Repair family	Defect it targets	How it changes the instance
Safe pair merge	Two existing components can be joined without atom overlap or tree-edge conflict.	Tests pairwise unions of incumbent components and accepts only resource-disjoint valid merges. This is the lowest-risk repair because it changes only two components at a time.
High-margin split and repack	A small score-sensitive instance may contain a component boundary that is locally wrong even though the whole forest is valid.	Opens a bounded region, tries alternative local partitions, and keeps the replacement only when the recombined forest has fewer components.
Common-cluster refinement	The current forest may cut across clusters that are still common or nearly common in both trees.	Builds regions around shared cluster boundaries and tries to replace several small components by fewer validated components aligned with that shared structure.
Structured split repack	The current components are valid individually but jointly inefficient inside a compact region.	Uses metafeature gates to choose split-style or edge-style repack neighborhoods, then re-solves those regions with the same atom and edge-resource constraints.
Order-based forest merge	Adjacent components in one or both leaf orders are likely to be near one another in the trees.	Scans short runs of current components in leaf order, validates their unions, and merges them if the union uses no resource already needed by the rest of the forest.
Selected-feedback refinement	The packer may have selected promising components whose neighborhoods were not fully represented in the candidate pool.	Adds new candidates around the current selected masks, solves the updated pool, and repairs the resulting partition before comparing it to the incumbent.
Near-clade and guarded-triple refinement	Useful local components may be close to clades or supported by triplets even when exact clusters are absent.	Adds near-clade masks or topology-compatible triple masks, packs them, and accepts only after the standard repair ladder validates a smaller partition.
Edge-conflict repack	A candidate merge is attractive, but it is blocked by components that already use the same tree edges.	Identifies the blocking components, forms a bounded region containing the merge and its blockers, and rebuilds that region with fewer edge conflicts.
Counterfactual blocker repack and LNS	A greedy incumbent can be trapped by a small number of blocking components.	Temporarily destroys a bounded region, including blockers, rebuilds it under a local time cap, optionally performs small follow-up merges, and accepts only if the whole forest improves.

Table 3: Repair mechanisms grouped by the kind of residual defect they are meant to fix. All repairs share the same acceptance rule: no invalid replacement and no component-count regression.

Instance bucket	Cases	Mean score gap	Gap sum	Component gap
$n \leq 200$	28	4.173	116.841	36
$201 \leq n \leq 700$	20	4.400	88.009	157
$701 \leq n \leq 2200$	16	3.864	61.822	299
$2201 \leq n \leq 6000$	8	2.017	16.138	215
$n \geq 6001$	8	0.843	6.741	511

Table 4: The score gap is largest in small and medium regions, even though the largest raw component gap appears on huge instances. This is why the guarded architecture restores the small/medium kernel but keeps the large branch simple.

Instance	Guard side	Leaves	Result	Journey summary
h70	small/medium	55	40 components	Parse two trees; choose the small branch; build small-solver tree arrays, dynamic-programming candidate masks, and a resource-packing model; keep the first completed 40-component forest as the incumbent.
h43	large single core	2453	1593 components	Parse two trees; reduce to 2301 atoms; compute metafeatures; build candidate-pool metadata; repair and pack local validated unions; expand selected atoms to original labels.

Table 5: Representative 300-second guarded-core journeys. The table describes which side of the guard is taken and what kind of runtime state is created along the way.

Instance	Stage	Working objects	Effect on the instance
h70	Input parsing and label mapping	Two rooted trees; one shared label order; leaf bit masks	Converts Newick text into a bit-mask representation used by every later stage.
h70	Guard decision	Leaf count $n = 55$	Chooses the score-sensitive small/medium path.
h70	Embedded small-kernel attempt	Compact node arrays; subtree-pair dynamic-programming tables; candidate masks; resource bitsets	Finds a complete 40-component partition.
h70	Small-side acceptance boundary	Best completed forest; component count	Keeps the best completed incumbent before any longer optional attempt.
h43	Input parsing and label mapping	Two rooted trees; one shared label order; leaf bit masks	Converts 2453 labels into a common integer-mask universe.
h43	Guard decision	Leaf count $n = 2453$	Chooses the large single-core path.
h43	Exact common-subtree reduction	Repeated annotations; common subtree masks; atom-to-original map	Contracts the working universe from 2453 leaves to 2301 atoms.
h43	Metafeature summary	Atom ratio; cluster Jaccard; local agreement signals	Sets local repair and packing behavior without changing the top-level route.
h43	Candidate-pool metadata	Reduced trees; annotations; leaf arrays; depth maps; ranks; validation cache	Creates the shared validity and resource-accounting context.
h43	Local repair and packing	Proposed atom-mask unions; validation records; used atom and edge masks	Replaces singleton-heavy partitions with compatible multi-atom components, reaching 1593 components.
h43	Output assembly	Selected atom masks; uncovered atoms; original labels	Expands atoms back to labels and emits singleton completions to form the final forest.

Table 6: Start-to-finish stage journey for the representative small and large instances. Counts refer to leaves before reduction, atoms after reduction, and components after packing.

Quantity	Value
Architectures compared	pure single core vs guarded core
Instances	h00, h05, h08, h25, h26, h43, h88
Seeds	20260501, 20260502
Paired blocks	14
Win/tie/loss blocks for guarded core	8 / 6 / 0
Mean score, previous architecture	92.731
Mean score, guarded architecture	96.258
Mean paired score difference	+3.527
Median paired score difference	+0.768
Total component-count difference	-107
Mean runtime difference	+39.571s
Wilcoxon signed-rank test	$p = 0.0078125$
Sign test	$p = 0.0078125$

Table 7: Paired statistical comparison on the score-proxy instances. Positive score differences and negative component-count differences favor the guarded architecture. Ties are retained in the practical win/tie/loss count and omitted from the signed-rank statistic.

Architecture / preset	Instances	Mean score	Worst score	Wall time
Pure single core, score proxy	h00, h05, h08, h25, h26, h43, h88	92.848	–	126.7s
Guarded core, score proxy	h00, h05, h08, h25, h26, h43, h88	96.258	92.303	441.9s
Guarded core, h41 proxy	h00, h05, h08, h25, h41, h43, h88	95.992	92.303	347.4s

Table 8: Quick-proxy results for the simplified fixed architecture.